به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین چهارم

مدرس: مهدی صفرنژاد — انتشار: ۲۴ فروردین ۹۹

درهم‌سازی

زمان تحویل این تمرین (شامل بخش نظری و بخش عملی) شنبه ۸ اردیبهشت ۹۹ ساعت ۱:۰۰ بامداد (یک ساعت پس از نیمه‌شب جمعه) می‌باشد. به توضیحات سایت درس دربارهٔ تمرین‌ها، تحویل آن و تأخیر توجه کنید. تمرین‌ها از سایت کوئرا تحویل گرفته می‌شوند. قسمت‌های نظری به صورت فایل PDF تحویل گرفته می‌شوند که این فایل یا باید با حروف‌چینی رایانه‌ای تولید شده باشد و یا اگر دست‌نوشته است، توسط نرم‌افزارهایی مثل CamScanner یا Office Lens و یا با دستگاه اسکنر تصویربرداری شده باشد.

سؤال ۱. [۱۰ نمره]

در گراف ساده nراسی و m یالی $s(v)$ را برابر مجموعه ی همسایه های راس v تعریف می‌کنیم. می‌خواهیم تعداد جفت راس هایی را بیابیم مانند u, v که $s(v) - \{u\} = s(u) - \{v\}$. الگوریتمی از $O(m + nlog(n))$ برای حل این مساله بیابید.

پاسخ: به ازای هر راس مجموعه راس هایش را به صورت مرتب شده در یک ارایه می‌نویسیم و$hash$ آنها را بدست می‌آوریم . سپس دوباره برای هر راس مانندv مجموعه همسایه هایش به همراه خود راسv را در یک آرایه می‌نویسیم و$hash$ آنها را بدست می‌آوریم. حال جواب مساله برابر است با تعداد جفت اعداد یکسان از$n * ۲$ عددی که در بالا بدست آورده ایم.

سؤال ۲. [۱۰ نمره]

یک رشته به طول n داریم، می‌خواهیم الگوریتمی برای یافتن تعداد اعداد طبیعی مانند k که $۱ <= k <= n$ و به ازای هر i داشته باشیم $s(i) = s(i + k)$ را بیابیم. الگوریتمی از $o(n)$ برای اینکار بدست بیاورید.

پاسخ: کافی است یکسان بودن دو رشته$s[۰, n - k]$ و$s[k, n]$ را به کمک$hash$ بررسی کنیم. درصورت یکسان بودن، شرط مساله برایk برقرار است.

سؤال ۳. [۱۲ نمره]

فرض کنید که می‌خواهیم مجموعه‌ی A را به مجموعه ی B نگاشت دهیم. احتمال آنکه دو عضو مشخص از A به یک عضو از B متناظر شوند را درنظر بگیرید. ثابت کنید زمانی این احتمال کمینه است که احتمال متناظر شدن هر دو عضو از A به هر عضو از B با یکدیگر برابر باشد.

پاسخ:

احتمال آنکه عنصر iام از A به عنصر jام از B نگاشته شود را $P(j)$ در نظر بگیرید. حال احتمال آنکه دو عضو از A به یک عضو j نگاشت داده شود برابر $P(j)^۲$ است . پس احتمال مورد نظر سوال برابر مجموع $P(j)^۲$ به ازای$۰ < j < n + ۱$ است. حال فرض کنید که$p(i)$ با$p(j)$ برابر نباشد . کافی است که جای این دو $(p(i) + p(j))/۲$را در نظر بگیرید و ببینید که مجموع فوق کم میشود. پس مساله اثبات می‌شود (چرا؟).

سؤال ۴. [۲۰ نمره]
یک خانواده H از توابع درهم سازی در نظر بگیرید که مجموعه متناهیU را به مجموعه متناهی B متناظر می‌کند. به این خانواده $e-callable$ می‌گوییم هرگاه برای تمام جفت های متمایز k,l از U داشته باشیم:
$Probability\{h(k)=h(l)\}\leq e$ به طوری احتمال، روی گرفتن یک تابع درهم سازی h به صورت رندوم از خانواده H درنظر گرفته شده است.ثابت کنید باید داشته باشیم:
$e\geq \frac{۱}{|B|}-\frac{۱}{|U|}$
که در آن $|A|$ تعداد اعضای مجموعه ی A است.(راهنمایی: از نتیجه‌ی مساله‌ی قبل استفاده کنید.)

پاسخ:

با توجه به مسئله قبل تعداد برخورد ها زمانی کمینه میشود که احتمال برخورد اعضا در B با یکدیگر برابر باشند. حال تعریف کنیم B نگاشته می‌شوند. پس $u_i=u/b$ برای هر $j\in B$ ju$_i$.
با توجه به نکته بالا برای هر تابع درهم سازی تعداد کل برخورد ها باید حداقل برابر $b\frac{(\frac{u}{b})(\frac{u}{b}-۱)}{۲}$ باشد. همچنین تعداد جفت اعضای متفاوت برابر است با $\frac{u(u-۱)}{۲}$. پس بنابراین تعداد برخورد ها به ازای هر جفت متمایز باید حداقل برابر مقدار زیر باشد:

$$\frac{b(\frac{u}{b})(\frac{u}{b}-۱)/۲}{u(u-۱)/۲}=\frac{\frac{u}{b}-۱}{u-۱}$$
$$>\frac{\frac{u}{b}-۱}{u}=\frac{۱}{b}-\frac{۱}{u}.$$

پس برای کران ϵ برای احتمال برخورد هر جفت متمایز از اعضا نمی‌تواند کمتر از $\frac{۱}{|B|}-\frac{۱}{|U|}$ باشد.

سؤال ۵. [۸ نمره]
تمام زیر آرایه های یک آرایه را بیابید که جمع آنها برابر ۱ شود! زیر آرایه به تعدادی عنصر متوالی از یک ارایه می‌گویند (می‌تواند شامل کل آرایه شود). این کار را در$o(n)$ انجام دهید .

پاسخ: ایده این است که مجموع اعضای آرایه از ابتدا تا هر index را در یک دیکشنری ذخیره کنیم . حال برای اینکه چک کنیم که یک زیر آرایه با مجموع ۱ وجود دارد ، برای هر i index که جمع اعضا تا آن index برابرx است ،ایا index دیگر که کمتر از i باشد با جمع$x - ۱$ وجود دارد یا خیر .

سؤال ۶. [۱۸ نمره]
n درخت باینری داریم که هر کدام k راس دارند.عملیات تعویض یعنی دو زیر درخت یک راس از یک درخت را با یکدیگر عوض کنیم. دو درخت قابل تبدیل شدن به یکدیگرند اگر و تنها اگر با تعدادی عملیات تعویض بتوان از یک درخت ، درخت دیگر را ساخت. الگوریتمی کارا از $O(nk)$ برای یافتن "تعداد" جفت درخت هایی که قابل تبدیل به یکدیگرند ارائه دهید.

پاسخ: درخت ها را به صورت زیر در هم سازی می‌کنیم:

فرض کنید $H(v)$ درهم سازی درختی ریشه‌دار از v باشد و v دو تا بچه L و R دارد. کاری که می‌کنیم دو تا بچه‌را بر اساس تابع درهم سازی‌شان مرتب می‌کنیم، یعنی برای مثال اگر زیر درخت سمت چپ تابع درهم سازی اش از سمت راستی بزرگتر بود آن‌ها را جابه‌جا می‌کنیم. فرض کنید L' و R' ترتیب مرتب‌شده این‌دو باشد.

$$H(v) = asciicode('(') \times mod^{(}size(L') + size(R')) + H(L') \times mod^{(}size(R) + ۱) + asciicode(')')$$

مقدار $H(v)$ اگر رأس v وجود نداشته‌باشد برابر صفر است، همچنین $size(v)$ بیانگر اندازه زیر درخت v است. توجه کنید تابع درهم سازی بالا به نوعی درخت را متناظر یک پرانتزگذاری می‌کند و پرانتزگذاری مورد نظر که یک رشته‌است از طریق روش درهم‌سازی معمول درهم سازی می‌شود. به سادگی دیده می‌شود دو درخت قابل تبدیل به هم دقیقا یک مقدار می‌گیرند.

توجه کنید پس از درهم‌سازی کافیست یک آرایه مثل cnt_h داشته‌باشیم که تعداد درخت‌هایی را بشمارد که مقدار تابع درهم سازی آن برابر h است. سپس جواب مسئله برابر مقدار زیر خواهد بود:

$$\sum_{h \in R_h} \binom{cnt_h}{۲}$$

محاسبه مقدار درهم سازی هر درخت با اردر تعداد رئوس درخت یعنی $O(n)$ امکان‌پذیر است پس محاسبه مقدار بالا برابر $O(nk + mod)$ است که اگر اندازه mod را مقداری منطقی انتخاب کنیم و یا درهم سازی مناسبی انتخاب کنیم الگوریتمی کارا است.

توجه کنید می‌توانید mod را بزرگ تعریف کنید و مقادیر مختلف درهم‌سازی را در یک درخت جست‌وجوی دودویی بریزید و مقدار cnt_h را با $O(log_۲ n)$ بشمارید. این هم مورد قبول است. هر تابع درهم سازی مناسب دیگر که جواب‌گو باشد برای درخت‌ها نیز جوابگو خواهد بود.

سؤال ۷. [۱۲ نمره]
فرض کنید یک جدول درهم سازی با m خانه داریم که خانه‌های آن را با اندیس‌های $\{۰, ۱, ..., m - ۱\}$ شماره گذاری کرده‌ایم.برای

یافتن عنصر با کلید k در این جدول مطابق الگوریتم زیر عمل می‌کنیم.

۱.شمارنده‌های $h(k) \to x$ و $۰ \to y$ را مقدار دهی کن.

۲.اگر کلید k در خانه xام جدول بود یا این خانه خالی بود جست‌وجو را متوقف کن.

۳.x و y را به شکل $y + ۱ \to y \ mod \ m$ و $(x + y) \to x \ mod \ m$ به‌روز کن و به مرحله‌ی ۲ برگرد.

آ. [۴ نمره] ثابت کنید الگوریتم فوق یک نوع وارسی درجه ۲ است.

ب. [۸ نمره] حال فرض کنید $m = ۲^d$ که d یک عدد صحیح مثبت است. ثابت کنید الگوریتم فوق در بدترین حالت تمام خانه‌های جدول را بررسی می‌کند.

پاسخ: آ) در مرحله iاُم اجرای الگوریتم فوق خانه‌ی وارسی شده را می‌توان به شکل زیر به‌دست آورد.

$$h(k,i) = x(i) = (h(k) + ۱ + ۲ + ... + i + ۱) \mod m = (h(k) + \frac{i(i+۱)}{۲}) \mod m$$

بنابراین این الگوریتم، یک الگوریتم وارسی درجه دو است.

ب) حالتی که جدول پر است را در نظر بگیرید. کافیست ثابت کنیم تابع $h(k,i)$ با دامنه $۰ \leq i \leq ۲^d - ۱$ تابعی یک به یک است. (چرا؟) بنابراین فرض کنید i و j به گونه‌ای هستند که $h(k,i) = h(k,j)$. در این صورت

$$h(k,i) = h(k,j) \implies \frac{i(i+۱)}{۲} \equiv \frac{j(j+۱)}{۲} \pmod{۲^d} \implies (i-j)(i+j+۱) \equiv ۰ \pmod{۲^{d+۱}}$$

از طرفی $i \leq ۲^d - ۱$ و $j \leq ۲^d - ۱$ بنابراین $i + j + ۱ \leq ۲^{d+۱} - ۱$. بنابراین $i + j - ۱$ بر $۲^{d+۱}$ بخش‌پذیر نیست. بنابراین

$$i - j \equiv ۰ \pmod{۲^{d+۱}}$$

که با توجه به اینکه i و j هر دو کمتر از ۲^d هستند از عبارت فوق می‌توان نتیجه گرفت $i = j$. بنابراین خواسته ما اثبات شد.

پاسخ:

سؤال ۸. [۱۰ نمره]
آرایه‌ی A از اعداد طبیعی داده شده است.می‌دانیم هر عدد طبیعی حداکثر ۱۰۰۰ بار در آرایه آمده است.
دو نوع کوئری داریم:

۱.کوئری نوع اول این است که برای هر دو عدد طبیعی n, r جایگاه r امین n در دنباله را بگوییم.

۲.کوئری نوع دوم این است که عضوی مانند i را در دنباله عوض کرده و j را قرار دهیم.(تضمین می‌شود این تغییر شرط اولیه آرایه را نگه می‌دارد.)

با استفاده از درهم سازی داده ساختاری طراحی کنید که جواب هر پرسش را در $O(۱)$ بدهد.

پاسخ: جواب مسئله یکتا نیست اما یکی از جواب هایی که این پیچیدگی زمانی را به ما می دهد داده ساختار زیر است:

از یک جدول درهم سازی زنجیره ای استفاده می کنیم و روی هر عضو لینک لیست های جدول یک پوینتر به یک $AVL\ TREE$ قرار می دهیم که در هر $AVL\ TREE$ جایگاه های عدد مربوط به آن را قرار می دهیم.

در این صورت برای هر پرسش نوع اول در $O(۱)$ درخت راس پیدا می شود و چون هر درخت حداکثر ۱۰۰۰ راس دارد در $O(۱)$ می توانیم پاسخ پرسش را بدهیم.

به همین صورت چون اضافه کردن و حذف کردن از درخت ها در زمان $O(۱)$ انجام می شود پرسش دوم هم در زمان مطلوب قابل انجام است.

در پاسخ زمان پیدا کردن یک درخت راس را $O(۱)$ در نظر گرفتیم چون نسبت اندازه جدول و اندازه آرایه را میتوان عددی ثابت گرفت و بطور احتمالی طول هر لینک لیست ثابت است.

سؤال ۹. [۲۰ (امتیازی) نمره]
یک جدول درهم سازی از اندازه‌ی m در نظر بگیرید که n آیتم را در خودش ذخیره کرده است که$n \leq m/۲$. برای رفع مشکل برخود نیز از روش آدرس دهی باز استفاده شده است. فرض کنید متغیر تصادفی x_i نمایانگر تعداد جستجو های موردنیاز برای i امین درج کردن باشد. فرض کنید متغیر تصادفی X برابر بیشینه مقدار x_i ها باشد. ثابت کنید که $E[X]$ از مرتبه‌ی $O(log(n))$ است. (راهنمایی : ثابت کنید $probability\{X > ۲log(n)\} \leq \frac{۱}{n}$ سپس از این موضوع در محاسبه‌ی $E[X]$ و تخمین های مربوطه استفاده کنید .)

پاسخ: اگر متغیر تصادفی X نمایانگر تعداد جستجو های برای یافتن کلید در یک سری جستجوی ناموفق باشد آنگاه $Pr\{X \geq i\} \leq \alpha^{i-۱}$ (چرا؟). چون $n \leq m/۲$ پس داریم: $\alpha \leq ۱/۲$. حال i را برابر $k+۱$ قرار می‌دهیم پس داریم:

$$Pr\{X > k\} = Pr\{X \geq k+۱\} \leq (۱/۲)^{(k+۱)-۱} = ۲^{-k}.$$

حال k را برابر $۲logn$ قرار می‌دهیم، پس احتمال اینکه در i امین درج به بیش از $k = ۲logn$ جستجو نیاز باشد حداکثر برابر است با:

$$۲^{-۲logn} = (۲^{logn})^{-۲} = n^{-۲} = ۱/n^{۲}.$$

فرض کنید که A رویداد $X > ۲logn$ باشد و برای $i = ۱, ۲, ..., n$ فرض کنید رویداد A_i رویداد این باشد که $X_i > ۲logn$.
در قسمت بالا نشان دادیم که $Pr\{A_i\} \leq ۱/n^{۲}$ برای هر i . حال توجه کنید که
$Pr\{A\} \leq Pr\{A_۱\} + Pr\{A_۲\} + ... + Pr\{A_n\}$ چرا که A اشتراک A_i ها است. پس این عبارت حداکثر برابر است با :

$$Pr\{A\} \leq n.\frac{۱}{n^{۲}} = ۱/n.$$

حال با توجه تعریف امید ریاضی داریم :

$$E[X] = \sum_{k=۱}^{n} k.Pr\{X = k\}$$
$$= \sum_{k=۱}^{[۲logn]} k.Pr\{X = k\} + \sum_{k=[۲logn]+۱}^{n} k.Pr\{X = k\}$$
$$\leq \sum_{k=۱}^{[۲logn]} [۲logn].Pr\{X = k\} + \sum_{k=[۲logn]+۱}^{n} n.Pr\{X = k\}$$

$= [۲logn] \sum_{k=۱}^{[۲logn]} Pr\{X = k\} + n \sum_{k=[۲logn]+۱}^{n} Pr\{X = k\}.$

حال برای عبارت بالا جای جمع اول عدد ۱ و جای جمع دوم با توجه به نکات ذکر شده در بالا عبارت $۱/n$ را تخمین می‌زنیم داریم:

$E[x] \leq [۲logn]/۱ + n.(۱/n)$
$= [۲logn] + ۱$
$= O(logn).$

سؤال ۱۰. [۲۵ نمره]

دانیال قصد دارد در یک مسابقه‌ی تلویزیونی به صورت تلفنی شرکت کند، در این مسابقه تعداد زیادی عدد وجود دارد، و فرد پس از دیدن این اعداد، باید به سوالات گزارشگر درباره‌ی این اعداد پاسخ دهند. هر سوال گزارشگر به صورت یک عدد است و دانیال باید بگوید که آیا در بین اعداد سه عدد وجود دارد که جمع آن‌ها برابر عدد داده‌شده شود یا نه.

به دانیال کمک کنید که برنده شود تا سهمی از جایزه‌اش را به شما بدهد.

ورودی:
شامل یک لیست از اعداد است که در یک خط آمده‌اند. اعداد می‌توانند منفی باشند و اندازه‌ی آن‌ها کمتر از 10^{10} می‌باشد.

سپس یک عدد در خط بعدی می‌آید که نمایانگر تعداد سوالات گزارشگر است.

سپس در هر خط یک عدد می‌آید که یک سوال گزارشگر می‌باشد.

خروجی:
به ازای هر سوال گزارشگر اگر به صورت جمع سه عدد نوشته می‌شد، در یک خط متن YES وگرنه NO را چاپ کنید.

محدودیت‌ها:
استفاده از داده ساختار dictionary پایتون مجاز نمی باشد. برای مقایسه رشته ها اجازه استفاده از عملگر == را ندارید. همچنین اجازه استفاده از داده ساختارهایی که در کلاس درس تدریس نشده اند را ندارید.

ورودی و خروجی نمونه:

تست اول

stdin	**stdout**
1 2 1000 2000	NO
12	NO
4	NO
5	NO
6	NO
7	NO
2	YES
1	NO
1003	YES
3003	YES
3002	YES
3001	YES
2003	NO
3000	

پاسخ:

ایده‌ی اصلی این سوال ذخیره‌ی جمع جفت عددها در یک جدول درهم‌سازی و سپس به ازای هر کوئری q جست‌وجو کردن $q - arr[i]$ در این جدول است که $arr[i]$ برابر iمین عدد داده‌شده بین اعداد گزارشگر است. چالشی که در این زمینه وجود دارد، اطمینان از متمایز بودن اعداد است.

برای حل این مشکل کافیست به ازای هر جفت عدد یک آرایه به جمع آن‌ها در جدول درهم سازی نسبت دهیم و در آن همه‌ی جفت اندیس‌های با آن جمع را نگه داریم. همه‌ی جمع جفت عددها و اندیسشان را در یک hashset ذخیره کنیم. (برای هش کردن فقط از جمع استفاده می‌کنیم) هر عملیات ذخیره‌سازی از $O(۱)$ و تعداد این عملیات‌ها از $O(n^۲)$ می‌باشد.

برای هر کوئری‌ها نیز با حالت بندی کردن عدد سوم c مقدار $q - c$ را در این جدول جست‌وجو کرده و با مقایسه‌ی اندیس‌های ذخیره‌شده از متمایز بودن سه عدد مطمئن می‌شویم. به راحتی می‌توانید مشاهده کنید که اگر تعداد اندیس‌های ذخیره شده به ازای یک جمع بیشتر از یک باشد، آنگاه حتما سه عدد متمایز پیدا می‌شود. پس به ازای هر کوئری می‌توان در $O(n)$ معتبر بودن آن را بررسی کرد.

کد نمونه (با تشکر از کیان باختری):

```
def find_triplet(array, hash_table, target):
    for i in range(len(array)):
        local_target = target - array[i]
        index = ((local_target * 12203) + 8171) % 500000
        if index < 0:
            index = -index
        if hash_table[index] is False:
            continue
        else:
            for element in hash_table[index]:
                if element[0] != i and element[1] != i:
                    sum_of_these = array[element[0]] + array[element[1]]
                    if sum_of_these == local_target:
                        return 'YES'
    return 'NO'


def build_hash_table(array):
    array_size = len(array)
    hash_table = [False for _ in range(500000)]
    for i in range(array_size):
        for j in range(i + 1, array_size):
            sum = array[i] + array[j]
            index = ((sum * 12203) + 8171) % 500000
```

```
            if index < 0:
                index = -index
            if hash_table[index] is False:
                hash_table[index] = [(i, j)]
            else:
                hash_table[index].append((i, j))
    return hash_table


array = list(map(int, input().strip().split()))
hash_table = build_hash_table(array)
n = int(input())
for _ in range(n):
    target = int(input())
    print(find_triplet(array, hash_table, target))
```

سؤال ۱۱. [۴۰ نمره]

جیسون که یکی از مأموران اطلاعاتی سازمان سیا است به مأموریتی در کشوری آسیایی فرستاده شده است. هدف او از این مأموریت پیداکردن طعم آدامس مورد علاقه رئیس جمهور است. جیسون پس از انجام این مأموریت قصد دارد تا این طعم را به صورت مخفیانه برای سازمان سیا ارسال کند. اما چون رمزنگار بسیار ضعیفی است تصمیم می‌گیرد که این طعم را در دو رشته تصادفی از حروف انگلیسی جاگذاری کند (طعم آدامس را به صورت پیوسته در نقطه‌ای تصادفی در دو رشته قرار می‌هد.) و تضمین می‌کند که دو رشته هیچ زیررشته مشترکی با طول بیشتر مساوی از طول طعم آدامس ندارند و آن دو رشته را برای سازمان سیا با ایمیل ارسال کند. خوشبختانه این راه رمزنگاری او لو می‌رود و سعید قصد دارد این طعم را بیابد. سعید موفق شده است که ایمیل جیسون را هک کند. سعید نیاز به برنامه‌ای دارد با دادن دو رشته طعم آدامس رئیس جهمور را بدست بیاورد. به سعید کمک کنید تا این طعم را بیابد.

ورودی:

دو رشته که همان دو رشته‌ای هستند که جیسون برای سازمان سیا ارسال کرده است. طول دو رشته حداکثر ۵×۱۰^۳ است.

خروجی:

طعم آدامس مورد علاقه رئیس جمهور که در دو رشته جاگذاری شده است.

محدودیت‌ها:

رشته‌ها از حروف کوچک و بزرگ انگلیسی تشکیل شده‌اند. استفاده از داده ساختار dictionary پایتون مجاز نمی باشد. برای مقایسه رشته ها اجازه استفاده از عملگر == را ندارید. همچنین اجازه استفاده از داده ساختارهایی که در کلاس درس تدریس نشده اند را ندارید.

ورودی و خروجی نمونه:

تست اول

stdin	**stdout**
ssdcmsdersddlimooodmqccsernmf dsflimoosmcneerbfdser	limoo

پاسخ:

اگر بیشینه‌ی طول رشته‌ها را n بگیریم، ابتدا برای هر یک از رشته‌ها به کمک الگوریتم hash rolling در $O(n)$ جدول هش مربوط به i کارارکتر اول رشته را تشکیل می‌دهیم. پس از ساخت این آرایه می‌توانیم که مقدار هش هر زیربازه‌ای از این رشته را در $O(۱)$ بیابیم.

حال روی طول بزرگترین زیررشته‌ی مشترک جست‌وجوی دودویی می‌کنیم، با در نظر گرفتن طول l برای بررسی در جست‌وجو، همه‌ی l در رشته‌ی اول را در $O(n-l)$ در یک جدول درهم‌سازی ریخته، سپس همه‌ی زیر رشته‌های به طول l از رشته‌ی دوم را در این جدول جست‌وجو می‌کنیم. از آنجایی که هر جست‌وجو از $O(۱)$ است، روی هم $O(n-l)$ هزینه کرده‌ایم. حال اگر یک از زیررشته‌های رشته‌ی دوم در جدول موجود بود به این معنی است که طول بزرگترین زیررشته بیشتر از l است، l را افزایش می‌دهیم در غیر این صورت کاهش می‌دهیم.

کد نمونه (با تشکر از ارشیا سلطانی) :

```python
class HashString:
    def __init__(self, s):
        self.mod = int(1e18 + 7)
        self.base = int(373)
        self.s = s
        self.pre = [0] * (len(s) + 1)
        self.tav = [0] * (len(s) + 1)
        self.tav[0] = 1
        self.lens = len(s)
        for i in range(1, len(s) + 1):
            self.pre[i] = (self.pre[i - 1] * self.base + ord(s[i - 1])) % self.mod
            self.tav[i] = (self.tav[i - 1] * self.base) % self.mod

    def get_hash(self, l, r):
        return ((self.pre[r + 1] - self.tav[r - l + 1] * self.pre[l]) % self.mod +
    self.mod) % self.mod


class HashTable:
    def __init__(self):
        self.mod=10003
        self.arr=[None]*self.mod
    def add(self,v):
        k=v%self.mod
        if self.arr[k] is None:
            self.arr[k]=[]
        self.arr[k].append(v)
    def find(self,v):
        k=v%self.mod
        if self.arr[k] is None:
            return False
        for u in self.arr[k]:
            if u==v:
                return True
        return False


def findGood(a, b, tol):
    x = HashTable()
    for i in range(0, a.lens - tol + 1):
        x.add(a.get_hash(i, i + tol - 1))
```

```
    for i in range(0, b.lens - tol + 1):
        if x.find(b.get_hash(i, i + tol - 1)):
            return (i, i+tol)
    return None



s = input()
t = input()
a = HashString(s)
b = HashString(t)

beg = 0
en = max(len(s), len(t)) + 1

result = None

while en - beg > 1:
    mid = (en + beg) // 2
    match = findGood(a, b, mid)
    if match is not None:
        beg = mid
        result = match
    else:
        en = mid

if result:
    print(s[result[0]:result[1]])
```

سؤال ۱۲. [۴۰ نمره]

سروش و علیرضا و مهدی و اسماعیل قصد دارند بازی چهارنفره‌ای را انجام دهند، در این بازی یکی از کارت‌ها بسیار محبوب است و همه قصد دارند این کارت را بردارند برای اینکه در حق کسی بی‌انصافی صورت نگیرد، تصمیم می‌گیرند که برای اخذ این کارت به صورت تصادفی عمل کنند.

متاسفانه چون علیرضا که در اینگونه بازی‌ها آوازه‌ی خوبی ندارد، افراد برای جلوگیری از تقلب، تصمیم می‌گیرند به گونه‌ی عجیبی یک نفر را به تصادف انتخاب کنند. چهار رشته‌ی بزرگ عدد درست کرده و هر کس یکی از آن‌ها را انتخاب کند. فردی که بزرگترین زیررشته‌ی پالیندروم در رشته‌اش موجود باشد، برنده است.

یک رشته را پالیندروم گوییم اگر از دو طرف به یک شکل خوانده شود. در فارسی کلمه‌ی داماد و در انگلیسی کلمه‌ی testset نمونه‌هایی از کلمات پالیندروم هستند.

حال علیرضا که به این سادگی‌ها حاضر نیست بیخیال برنده شدن شود، از شما می‌خواهد کدی بزنید که با گرفتن یک رشته‌ی ورودی از حروف کوچک انگلیسی اندازه‌ی بزرگترین زیررشته‌ی پالیندروم را بگوید.

ورودی:
یک رشته با طول حداکثر ۵×۱۰^۴ است. اجزای رشته جروف کوچک انگلیسی هستند.

خروجی:
یک عدد که اندازه‌ی بزرگترین زیررشته‌ی پالیندروم در رشته‌ی ورودی می‌باشد.

محدودیت‌ها:
استفاده از داده ساختار dictionary پایتون مجاز نمی باشد. برای مقایسه رشته ها اجازه استفاده از عملگر == را ندارید. همچنین اجازه استفاده از داده ساختارهایی که در کلاس درس تدریس نشده اند را ندارید.

ورودی و خروجی نمونه:

تست اول

stdin	**stdout**
pqhlettelm	6

پاسخ:

ابتدا به کمک الگوریتم hash rolling پیش پردازشی از $O(n)$ انجام می‌دهیم و مقادیر هش مربوط به رشته و معکوس آن را نگه می‌داریم. پس از این به بعد می‌توانیم در $O(۱)$ هش مربوط به یک رشته و هش معکوس آن را بیابیم. پس می‌توانیم در $O(۱)$ بررسی کنیم که آیا هش یک رشته با هش معکوس آن برابر است، و پالیندروم بودن آن را تشخیص بدهیم.

برای یافتن بزرگترین زیررشته‌ی پالیندروم، ابتدا مرکز رشته را تعریف می‌کنیم: اگر طول یک رشته l فرد باشد حرف $\frac{l+۱}{۲}$ و اگر زوج باشد، دو حرف $\frac{l}{۲}$ و $\frac{l}{۲}+۱$ را مرکز رشته می‌نامیم.

حال توجه کنید که اگز یک زیررشته‌ی پالیندروم داشته باشیم، تمام زیررشته‌های این زیررشته با همان مرکز نیز پالیندروم می‌باشند. لذا برای هر مرکز، با استفاده از جست‌وجوی دو دویی روی طول، بزرگترین زیررشته به آن مرکز را در $O(\log(n))$ بیابیم.

از آنجایی که تعداد مراکز زوج و فرد در بین زیررشته‌های یک رشته از $O(n)$ است، با حالت بندی روی مرکز، بزرگترین زیررشته‌ی به آن مرکز را پیدا کرده و بیشینه میگیریم.

کد نمونه:

```
import math

MOD = 1000 * 1000 * 1000 + 7
BASE = 101

MAX_SIZE = 100000

BASELIST = [0] * (MAX_SIZE + 1)
BASELIST[0] = 1
for i in range(1, MAX_SIZE + 1):
    BASELIST[i] = (BASELIST[i - 1] * BASE) % MOD

def make_hash(s):
    hashlist = [0] * (len(s) + 1)
    hashlist[0] = 0
    for i in range(1, len(s) + 1):
        hashlist[i] = (hashlist[i - 1] * BASE + ord(s[i - 1]) - ord('a')) % MOD
    return hashlist

def get_hash(hashlist, src, des):
    return (hashlist[des+1] - hashlist[src] * BASELIST[des - src + 1] + MOD) % MOD



def find_biggest_palindrom(s):
    hashlist = make_hash(s)
    rev_hashlist = make_hash(s[::-1])
    biggest_palindrom_size = 0
    for odd_center in range(len(s)):
        end_radius = min(len(s) - odd_center - 1, odd_center)
        start_radius = 0
        while start_radius != end_radius:
            r = int(math.ceil((start_radius + end_radius) / 2))
            if get_hash(hashlist, odd_center-r, odd_center+r) == get_hash(rev_hashlist
    , len(s)-1-r-odd_center, len(s)-1+r-odd_center):
```

```
                start_radius = r
            else:
                end_radius = r - 1
        biggest_palindrom_size = max(biggest_palindrom_size, 2*start_radius+1)

    for even_center in range(len(s) - 1):
        end_radius = min(len(s) - even_center - 1, even_center+1)
        start_radius = 0
        while start_radius != end_radius:
            r = int(math.ceil((start_radius + end_radius) / 2))
            if get_hash(hashlist, even_center-r+1,even_center+r) == get_hash(
    rev_hashlist, len(s)-1-r-even_center, len(s)-1+r-even_center-1):
                start_radius = r
            else:
                end_radius = r - 1
        if start_radius == 0:
            continue
        biggest_palindrom_size = max(biggest_palindrom_size, 2*start_radius)
    return biggest_palindrom_size

if __name__ == '__main__':
    print(find_biggest_palindrom(input()))
```

موفق باشید